

Coding & Solution

Date	01 October 2026
Team ID	6ab7f752d083b0e2ee72b0bf
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	5 Marks

Coding & Solution

This section summarises the implemented code and overall solution for PocketSmart AI: a Gemini-powered budget planning web application built with Flask.

Solution Summary

Repository Link / URL	https://github.com/Saran2007-prog/PocketAI
Programming Language(s)	Python, JavaScript, HTML, CSS (Jinja2 templates)
Framework(s) Used	Flask 3.1.2, Flask-SQLAlchemy 3.1.1, Flask-Login 0.6.3, Google Gemini API through the google-genai SDK (default model gemini-3.8-flash)
Key Features Implemented	• Register, login and logout with hashed passwords and Flask-Login sessions (/register, /login, /logout) • Home Interior planner (/planner/home) • Party planner (/planner/party) • Jewelry planner with optional outfit image (/planner/jewelry) • Gemini JSON plan: allocations, recommendations, tips and disclaimer • Platform search links for Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO and Myntra • Dashboard with the latest 10 plans and saved plan page (/dashboard, /history/<id>) • Input validation, upload checks (type, 5 MB limit) and /api/health
Pending / Incomplete Features	• Live product and price data: platform results are search links, prices are AI estimates • CSRF protection, rate limiting and secure-cookie settings • Automated tests and a Dockerfile • Public deployment and a PostgreSQL database with migrations
Setup / Run Instructions	• Install Python 3.11+ and Git, clone the repository (git clone https://github.com/santhoshcreator/pocket-smart-AI.git) and open the project folder that contains run.py • Create and activate a virtual environment (python -m venv .venv) • Install dependencies: pip install -r requirements.txt • Copy .env.example to .env and set GEMINI_API_KEY, GEMINI_MODEL and SECRET_KEY • Start the server: python run.py • Open http://127.0.0.1:5000, register an account and use the planners

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial (detailed README; few inline comments)
4	Error handling is implemented for critical operations	Yes

S.No	Criteria	Status (Yes / No)
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes (GitHub: Saran2007-prog/ PocketAI)

Additional Notes / Comments:

The Gemini API key and SECRET_KEY are read from a .env file that is excluded by .gitignore. Recommendation links are search links generated by the application, not live platform data, and the plan prices are estimates that the user must verify. The Gemini model name is read from GEMINI_MODEL, so it can be changed without editing code.